

计算机真的能掷骰子吗？

对七种伪随机数生成器的统计学《测谎》实验

箬青寒

2026 年 8 月 3 日

摘要

掷骰子是人类最古老的随机体验，而计算机并不会真的掷骰子——它靠伪随机数生成器（PRNG）按确定性的公式吐出看似杂乱无章的数字。本文把这些生成器请上测谎席：对七种生成器，即 C 语言的 `C_rand`、Python 的 `Py_random`、手写线性同余生成器 `LCG_hand`、梅森旋转 `Mersenne`、圆周率 π 的小数位 `Pi`，以及两个故意作弊的正向对照 `Biased_Low` 与 `Short_LCG`，分别在样本量 10^4 、 10^5 、 10^6 下各掷 40 次骰子，用卡方均匀性检验查各面是否等概率，用 Wald-Wolfowitz 游程检验查前后是否独立。全部 1200 次运行、约 2400 次检验在本机约 10 秒内完成。结果有三点值得注意：两个作弊生成器被双双识破，表明测谎仪真的有检测力；短周期 LCG 的骗术卡方检验完全抓不到、游程检验一击命中，说明两种检验缺一不可；在本文的检测分辨率下， π 的小数位与诚实标杆一样清白，而取模偏差在 10^6 样本量内不可检测。实验还以 10000 次 p 值均匀性自检确认了检验代码本身可靠。本文的结论是：检验的分辨率决定能识破什么级别的作弊，双检验互补才能给生成器做可信的测谎。

关键词：伪随机数生成器；卡方检验；游程检验；拒绝率；置信区间

MSC 2020：62F03；62G10；65C10

目录

1	引言	2
2	相关工作	2
3	问题设定与记号	3
4	方法	4
5	实验与结果	5
6	讨论与局限	9
7	结论	9

1 引言

有人问，计算机真的能掷骰子吗？把这个问题换一种说法：计算机吐出来的一串随机数，敢不敢当成一把公平的骰子来用？赌场里的骰子可能灌了铅，程序里的骰子也可能藏着私心。本文想做的，是给几把程序骰子做一次统计学测谎。

随机数在生活里无处不在：抽奖、游戏、模拟、加密。多数时候它们由伪随机数生成器产生——一条确定性的公式，输入一个种子，输出一串看似杂乱、其实可以重演的数字。既然是公式算出来的，它们当然不是真随机；真正的问题是，它们的伪装像不像真的。一把被灌了铅的骰子，肉眼看不出来，得拿尺子称。

统计学就是那把尺子。本文的测谎仪由两项初等检验组成。卡方均匀性检验问第一问：六面出现的次数，像不像一颗公平骰子该有的样子？Wald-Wolfowitz 游程检验问第二问：这一把的结果，和上一把有没有扯上关系？两道关都过，才判诚实。

受测对象有七种生成器。Py_random 与 Mersenne 是公认的老实人，当标杆；C_rand、LCG_hand 与 Pi 是看起来可疑的普通选手；Biased_Low 与 Short_LCG 是故意做手脚的对照组——明知它们会作弊，正好用来检验测谎仪有没有失灵。另外还有一个低模数的演示生成器 Low_LCG，单独出场，负责解释分辨率的边界。

本文的贡献有三点。其一，把诚实度变成可以量化的指标：40 次重复下的拒绝率 ρ ，再配上 Wilson 95% 置信区间，谁诚实谁作弊一目了然。其二，用正向对照展示检验真有检测力，不是橡皮图章。其三，发现并解释了一个反直觉结果：短周期 LCG 的骗术让卡方检验完全落空，却逃不过游程检验的一击，说明两种检验缺一不可。

后文安排如下。第 2 节回顾相关文献，第 3 节给出记号与受测对象，第 4 节说明两项检验与显著性校正，第 5 节报告实验结果，第 6 节讨论局限与展望，第 7 节总结。

2 相关工作

随机性检验的开端要回溯到二十世纪初。Pearson [1] 提出卡方拟合优度检验，把这一串结果合理不合理的判断，变成一个可以查表的数；Fisher [2] 随后澄清了列联表与自由度的细节。针对序列本身的顺序结构，Wald 与 Wolfowitz [4] 提出游程检验，专门抓前后不独立。本文的两项检验，分别源自这两个传统。

伪随机数生成器的历史同样悠久。Lehmer [6] 提出线性同余生成器，递推式 $x_{n+1} = (a \cdot x_n + c) \bmod m$ 至今仍是入门教科书的标准道具；Marsaglia [8] 指出这类生成器的输出会落在低维超平面上，有名的“随机数主要落在平面上”就出自这篇短文；Park 与 Miller [9] 给出了一个最小标准。Knuth 的《计算机程序设计艺术》第二卷 [12] 系统整理了随机数与各类检验。Matsumoto 与 Nishimura [13] 提出的梅森旋转 MT19937，周期长达 $2^{19937} - 1$ ，是本文两个标杆生成器的内核。

现代检测工具已经发展成套的体检套餐。Marsaglia [10] 的 Diehard 与 L'Ecuyer 和 Simard [14] 的 TestU01 包含几十乃至上百项检验；NIST 的 SP 800-22 [15] 用 15 项检验为加密用途把关。套件化的趋势本身说明一个道理：单项检验容易漏网，组合才可靠。这与本文第 5 节的发现完全一致。

近年研究仍在推进。Foreman 等 [17] 分析检验套件，并用随机性抽取来改善生成器输出；Riesinger 与 Fuß [19] 发现 NIST 套件参考实现中的若干缺陷；Chai 等 [20] 综述了随机数在区块链与工业网络中的应用与检测。另有一支工作用神经网络学随机数：Okada 等 [16] 用生成对抗网络产出统计稳健的随机序列；Dahiya 等 [18] 则展示，对 PRNG 做极细微的篡改，能骗过标准的 NIST 检验。这些工作共同说明，测谎仍然是一场军备竞赛。

本文与前人的差别在取舍。大型套件追求全面，本文只用两项初等检验，面向高中生读者，把每个结论都建立在 40 次重复与预注册的显著性校正之上。方法简单，但每一句话都有数据兜底。

3 问题设定与记号

先约定记号。一个伪随机数生成器 G 由种子 s 驱动，输出一串整数或小数。本文统一先把输出变成 $[0, 1)$ 上的数 u ，再用同一把尺子骰子化。

定义 3.1 (骰子化). 对 $u \in [0, 1)$ ，骰子面定义为 $F = \lfloor 6u \rfloor + 1 \in \{1, \dots, 6\}$ 。若序列真的随机，每个面出现的概率都应是 $1/6$ ，样本量为 n 时每个面的期望频数都是 $E_i = n/6$ 。

对整数源的生成器， u 的取法有讲究。`C_rand` 用 $u = \text{rand}() / (\text{RAND_MAX} + 1.0)$ ，本机 $\text{RAND_MAX} = 32767$ ，分母取 32768 保证 $u < 1$ 不越界；`LCG_hand` 与 `Pi` 分别用 $u = x/m$ 与 $u = d/10^6$ 。只有第 5 节的取模演示改用 $F = (x \bmod 6) + 1$ 。前一种叫统一浮点映射，后一种叫取模映射，两种映射会引出关于分辨率的讨论。

指标方面，对一组固定的生成器与样本量，用 $K = 40$ 个种子各跑一次检验，记拒绝次数为 r ，则拒绝率 $\rho = r/K$ ，粒度为 0.025。每个 ρ 附 Wilson 95% 置信区间，公式见第 4 节。 ρ 明显高于 $\alpha = 0.05$ ，并不意味着这台生成器一定在作弊： ρ 偏高也可能是抽样波动——例如 `Mersenne` 在 $n = 10^5$ 时卡方 $\rho = 0.075$ ($3/40$) 即属此类——只有经多重校正后仍显著、且持续高于 α ，才判作弊。

受测对象列于表 1。两个标杆用于交叉验证，两个正向对照用于检验检测力，`Low_LCG` 是演示生成器，不进主矩阵。

表 1 受测生成器一览：出处、核心实现与在实验中的角色

生成器	出处与实现	角色
<code>C_rand</code>	Windows MSVC CRT 的 <code>rand()</code> ， $\text{RAND_MAX}=32767$	普通选手
<code>Py_random</code>	Python <code>random.random()</code> ，MT19937 封装	诚实标杆
<code>LCG_hand</code>	手写 LCG， $x_{n+1} = (1103515245x_n + 12345) \bmod 2^{31}$	普通选手
<code>Mersenne</code>	<code>numpy.random.Generator(numpy.random.MT19937(...))</code> ，MT19937	诚实标杆
<code>Pi</code>	圆周率 π 前 2.4×10^8 位小数，每 6 位一组	普通选手
<code>Biased_Low</code>	LCG 低 8 位， $u = (x \bmod 256)/256$ ，周期 ≤ 256	正向对照
<code>Short_LCG</code>	短周期 LCG， $m = 2^{10}$ ， $u = x/1024$	正向对照
<code>Low_LCG</code>	低模 LCG， $m = 2^5$ ， $u = x/32$	演示

为了避免测一百次总有好几次碰巧显著的误会，实验方案在跑数据之前就写死，这叫预注册。主比较族是 18 次配对 McNemar 检验，Bonferroni 校正后取 $\alpha^* = 0.05/18 \approx 0.0028$ ；生成器之间的整体差异用 Fisher–Freeman–Halton 精确检验；拒绝率随样本量的趋势用 Cochran–Armitage 检验，对主矩阵 7 家生成器各做一次、共 7 次，Bonferroni 校正后取 $\alpha^* = 0.05/7 \approx 0.0071$ 。游程一族只作描述性报告，不进校正预算。这些概念都在下一节说明。

把要检验的主张编成 H1–H7 七条假设，第 5 节逐条报告这些假设。H1：在统一映射下，各生成器的拒绝次数存在组间差异，Fisher–Freeman–Halton 精确检验应能把它分辨出来。H2：对每个生成器，拒绝率随样本量增大而上升，作弊者应呈现明显的上升趋势。H3 关于取模演示，下设两个子假设：H3a 以 `C_rand` 与 `Low_LCG` 在统一浮点映射下的拒绝率为对照基线；H3b 断言 `C_rand` 换成取模映射后，与自身统一映射的配对 McNemar 不显著，而 `Low_LCG` 因模数过低两种映射下都 40/40 拒绝。H4：存在这样的生成器，它让卡方检验全部通过、游程检验却大量拒绝，说明均匀性与独立性两关缺一不可。H5：`Pi` 在三个样本量下的拒绝率都贴近 α ，与标杆 `Py_random` 的配对比较无一显著，圆周率数字与诚实标杆一致。H6：未被刻意做手脚的生成器 `C_rand`、`LCG_hand`、`Mersenne`、`Pi` 与标杆 `Py_random`——即除正向对照与演示外的全部 5 家——拒绝率贴近 α ；其中 `C_rand`、`LCG_hand`、`Pi` 与标杆 `Py_random` 的配对比较在预注册 18 族内、经 $\alpha^* \approx 0.0028$ 校正后无一显著，`Mersenne` 与 `Py_random` 的配

对比较为族外描述性交叉验证（不入校正预算）；其诚实以”未被证伪”的形式成立。H7：正向对照 **Biased_Low** 在样本量足够大时被两项检验双双识破，配对 McNemar 极显著，证明测谎仪具有检测力。

4 方法

一次测谎分为六步，流程见下面的伪代码框。核心是两个统计检验：卡方均匀性检验与游程独立性检验。

一次测谎的标准流程

输入：生成器 G ，样本量 n ，种子 s 。

第 1 步 用种子 s 运行 G ，得到 n 个 $[0, 1)$ 上的数 $u_1, \dots, u_n$ ；

第 2 步 骰子化（定义 3.1）： $F_i = \lfloor 6u_i \rfloor + 1$ ，得到骰子面序列；

第 3 步 卡方检验：统计六面频数 $O_1, \dots, O_6$ ，按式 (4.1) 计算 χ^2 与单侧 p 值；

若 $p < 0.05$ ，记一次卡方拒绝；

第 4 步 游程检验：把每个面与 3.5 比较成大小符号串，数出游程 R ，按式 (4.2) 计算 z 与双侧 p 值；

若 $|z| > 1.96$ ，记一次游程拒绝；

第 5 步 换下一个种子，重复第 1-4 步，共 $K = 40$ 次；

第 6 步 输出拒绝率 $\rho = (\text{拒绝次数}) / 40$ 与 Wilson 95% 置信区间。

第一道关是卡方均匀性检验。把骰子面按 1 到 6 分成六类，统计观测频数 O_i ，与理论期望 $E_i = n/6$ 比较：

$$\chi^2 = \sum_{i=1}^6 \frac{(O_i - E_i)^2}{E_i}. \quad (4.1)$$

χ^2 的自由度是 $k - 1 = 5$ 。这一检验是单侧的：只有当六面计数偏离得离谱时 χ^2 才大， $p = P(\chi_5^2 \geq \chi_{\text{obs}}^2)$ 是右尾概率， $p < 0.05$ 就拒绝。直观地说，掷一颗公平骰子 600 次，某一面独得 150 次而其余五面平分，这个 χ^2 会大得可疑；六面差不多均匀， χ^2 就接近 0。

第二道关是游程独立性检验。先把序列转成二值：面大于 3.5 记 +，小于 3.5 记 -，因为骰子面没有恰好等于 3.5 的，所以不会出现平局。记 + 的个数为 n_+ 、- 的个数为 n_- ，连续同号的一段叫一个游程，总游程数记为 R 。在完全随机的假设下，

$$\mathbb{E}(R) = 1 + \frac{2n_+n_-}{n}, \quad \text{Var}(R) = \frac{2n_+n_-(2n_+n_- - n_+ - n_-)}{n^2(n-1)}, \quad (4.2)$$

检验统计量 $z = (R - \mathbb{E}(R)) / \sqrt{\text{Var}(R)}$ 近似服从标准正态分布，两侧都值得怀疑： $|z| > 1.96$ 就拒绝。这一检验是双侧的。直觉上，真随机的大小号应该像”大小大小”那样交错出现；如果出现”大大大大小小小小小”这种成串的局面，游程数偏少， z 会负得厉害，说明前后有关联。

比较两个生成器时，用配对 McNemar 检验 [5]。两个生成器共用同样的 40 个种子，把每个种子的拒绝情况配成对：两边都拒绝、两边都接受、只有一边拒绝。只有一边拒绝的种子才携带信息。当这种不一致对太少 ($b + c < 10$) 时，渐近公式不可靠，改用精确二项版本。

多重比较要校正。 $\alpha = 0.05$ 是单次检验犯错的概率；做 m 次检验，用 Bonferroni 把门槛压到 α/m ，整体错误率仍约 5%。本文主比较族共 18 次， $\alpha^* \approx 0.0028$ ，意味着 p 值要小于 0.0028 才算显著，比普通的 0.05 严苛得多。这 18 次的构成是固定的：主矩阵中除两个标杆外，**C_rand**、**LCG_hand**、**Pi**、**Biased_Low** 与 **Short_LCG** 各自与标杆 **Py_random** 配对，在 10^4 、 10^5 、 10^6 三个样本量下各做一次，共 $5 \times 3 = 15$ 次；再加上取模演示中 **C_rand** 取模映射对统一映射的三个样本量各一次（即 H3b 的 3 次），合计 18 次。这 18 次全部进入 Bonferroni 校正预算，**Low_LCG** 的演示比较不在其内。

H1 的 Fisher–Freeman–Halton 精确检验与 H2 的 Cochran–Armitage 趋势检验都只针对卡方检验的拒绝次数统计；游程检验的拒绝次数另作描述性报告，不进入这两族检验。因此 **Short_LCG** 的游程在 $n = 10^6$ 全部拒绝属于 H4 的描述性结论，不影响 H2 按卡方口径对趋势作出的判定。

关于置信区间，每个拒绝率 ρ 都报告 Wilson [3] 95% 置信区间：

$$\rho^{\pm} = \frac{\rho + \frac{z_{0.975}^2}{2K} \pm z_{0.975} \sqrt{\frac{\rho(1-\rho)}{K} + \frac{z_{0.975}^2}{4K^2}}}{1 + \frac{z_{0.975}^2}{K}},$$

其中 $z_{0.975} = 1.96$, $K = 40$ 。这个区间在 ρ 接近 0 或 1 时也不失真，适合小样本的比例。

注记 4.1. 统计检验只能证明没有抓到把柄，不能证明真的随机。本文说某生成器诚实，意思是它在 40 次重复、两个检验、给定样本量的范围内，表现得和标杆一样。

5 实验与结果

实验规模如下。主矩阵是 7 个生成器 $\times$ 3 个样本量 $\times$ 40 个种子，共 840 次运行，全部用统一浮点映射；取模演示是 3 组 $\times$ 3 $\times$ 40，共 360 次；合计 1200 次运行，每次同时做卡方与游程两项检验，约 2400 次检验。另有对标杆 `Py_random` 在 $n = 10^5$ 下重复 10000 次的 p 值均匀性自检。种子取 2024001–2024040， π 用 40 段互不重叠的数据窗口，每段 6×10^6 位。 π 的前 2.4×10^8 位小数取自公开的圆周率十进制数字库 (calculat.io)，首 30 位为 141592653589793238462643383279，可用来快速定位与抽查；各数字 0–9 的出现频率相对期望 2.4×10^7 的最大 $|z| = 1.73$ ，与均匀假设相符。BBP 公式 [11] 还提供在任意指定位快速生成 π 数字的独立校验途径。

运行环境是 Windows 11 家庭中文版，AMD Ryzen 7 8745H 八核十六线程，主频 3.8 GHz，内存约 20.3 GB，无独立 GPU。软件栈为 Python 3.12.9，配合 numpy 2.4.3、pandas 2.3.3、matplotlib 3.10.8 与 scipy 1.17.1。多进程下，1200 次实验约 10 秒完成。

正式解读结果前，先自证检验代码可靠，这是论文的前置闸门。G1 抽样分布校验：在标杆上重复 10000 次，卡方 p 值的 Kolmogorov–Smirnov 检验 $p = 0.7081$ ，游程 z 的 KS 检验 $p = 0.5165$ ，卡方经验拒绝率 0.0522，都与理论吻合；G2 均匀性自检：10000 个 p 值直方图的卡方拟合 $\chi^2 = 19.820$ ，自由度 19， $p = 0.4055$ ，KS 检验 $p = 0.1654$ ，没有系统性偏差（图 3）。G3–G5 的数据完整性、基线对照与结果签名也全部通过（每个格子的 40 次重复齐全、正向对照按预期被抓住、聚合数字与逐次记录一致）。闸门不过的结论不允许进入正文——这些数字是过了闸门的。

图 1 给出卡方拒绝率的鸟瞰图（右上角的局部放大把诚实组看得更清楚），表 2 给出卡方与游程两套逐格数字与 Wilson 95% 置信区间。先看两极：`Biased_Low` 的卡方柱在 $n \geq 10^5$ 像立柱一样直冲 1.0，`Short_LCG` 的卡方柱则贴地不动（它的问题在游程一列， $n = 10^6$ 时 40/40，见表 2），其余生成器全部贴着 $\alpha = 0.05$ 的虚线摆动。判断是，作弊者被抓住，老实人放行。

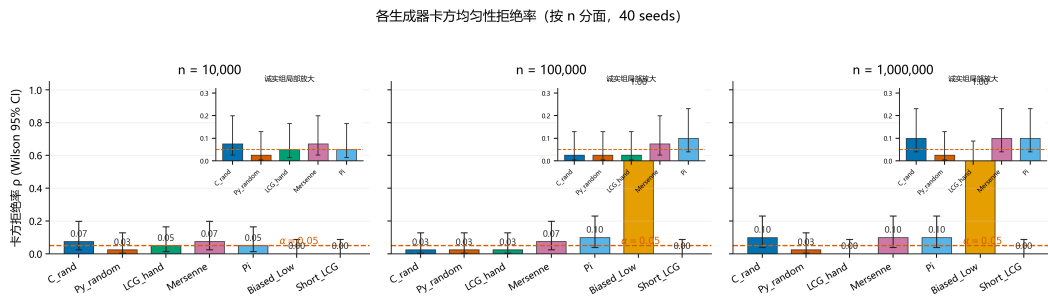


图 1 七种生成器在三个样本量下的卡方拒绝率 ρ (误差线为 Wilson 95% CI, 虚线为 $\alpha = 0.05$, 右上角为诚实组局部放大)

表 2 主矩阵结果：卡方与游程拒绝率 ρ 及其 Wilson 95% 置信区间（每格 40 次重复；粗体为关键结果）

生成器	n	卡方 ρ	卡方 95% CI	游程 ρ	游程 95% CI
Py_random	10^4	0.025	0.00–0.13	0.025	0.00–0.13
Py_random	10^5	0.025	0.00–0.13	0.050	0.01–0.17
Py_random	10^6	0.025	0.00–0.13	0.025	0.00–0.13
Mersenne	10^4	0.075	0.03–0.20	0.000	0.00–0.09
Mersenne	10^5	0.075	0.03–0.20	0.050	0.01–0.17
Mersenne	10^6	0.100	0.04–0.23	0.050	0.01–0.17
C_rand	10^4	0.075	0.03–0.20	0.050	0.01–0.17
C_rand	10^5	0.025	0.00–0.13	0.075	0.03–0.20
C_rand	10^6	0.100	0.04–0.23	0.025	0.00–0.13
LCG_hand	10^4	0.050	0.01–0.17	0.000	0.00–0.09
LCG_hand	10^5	0.025	0.00–0.13	0.050	0.01–0.17
LCG_hand	10^6	0.000	0.00–0.09	0.050	0.01–0.17
Pi	10^4	0.050	0.01–0.17	0.100	0.04–0.23
Pi	10^5	0.100	0.04–0.23	0.050	0.01–0.17
Pi	10^6	0.100	0.04–0.23	0.125	0.05–0.26
Biased_Low	10^4	0.000	0.00–0.09	0.000	0.00–0.09
Biased_Low	10^5	1.000	0.91–1.00	1.000	0.91–1.00
Biased_Low	10^6	1.000	0.91–1.00	1.000	0.91–1.00
Short_LCG	10^4	0.000	0.00–0.09	0.000	0.00–0.09
Short_LCG	10^5	0.000	0.00–0.09	0.000	0.00–0.09
Short_LCG	10^6	0.000	0.00–0.09	1.000	0.91–1.00

注：粗体标记两处关键结果——Biased_Low 在 $n \geq 10^5$ 两项检验均 40/40 拒绝（与 Py_random 的配对 McNemar $p = 1.17 \times 10^{-9}$ ）；Short_LCG 的卡方列三个 n 层全部 0/40 漏检，游程列在 $n = 10^6$ 为 40/40。

下面按假设逐条报告。先说组间差异（H1）。把每个样本量下 7 个生成器的卡方拒绝次数并成一张 7×2 表（7 行是生成器，2 列是拒绝与未拒绝），用 Fisher-Freeman-Halton 精确检验（三个样本量各做一次，按 $\alpha = 0.05$ 判定）。 $n = 10^4$ 时 $p = 0.457$ ，不显著； $n \geq 10^5$ 时 $p \approx 1 \times 10^{-5}$ ，显著，而且几乎完全由 Biased_Low 的 40/40 贡献。 $n = 10^5$ 时其余六家的卡方拒绝次数按表 2 的生成器顺序分别是 1、3、1、1、4、0，全在噪声范围内。整体检验只对真实缺陷开火，没有一堆噪音显著。

再看趋势（H2）。对每家生成器，把三个 n 层的拒绝比例做 Cochran-Armitage 趋势检验 [7]（以 $\log_{10} n$ 为剂量）。Biased_Low 的拒绝数从 0 跳到 40、40， $z = 9.49$ ， $p = 2.4 \times 10^{-21}$ ，在族内阈值 $\alpha^* = 0.05/7 \approx 0.0071$ 下显著；其余六家全部不显著， z 绝对值最大不超过 1.43。拒绝率随样本量上升，正是样本越多越能揭穿作弊的直接证据。

正向对照的检测力（H7）是整篇的关键标尺。Biased_Low 只取 LCG 的低 8 位，周期不超过 256，是刻意构造的灌铅骰子。 $n = 10^4$ 时它成功隐藏，0/40 拒绝，与标杆 Py_random 的配对 McNemar $p = 1.0$ ，同样不显著； $n = 10^5$ 起原形毕露，40/40 全拒绝，配对 McNemar 骤降至 $p = 1.17 \times 10^{-9}$ ，远低于 $\alpha^* \approx 0.0028$ 。检测力随样本量上升的轨迹，到这里就完整了。这一格的意义在于，测谎仪面对真作弊者确实会响，所以前面给诚实组的无罪判决才有底气。

双检验互补（H4）是全文最出彩的一格。Short_LCG 的周期只有 1024，数字分布均匀，却把相邻输出绑在同一个短周期上。卡方检验问的是各面是否等概率，它三个 n 层全部 0/40，完全漏检，与标杆 Py_random 的卡方配对 McNemar 在三个样本量下 $p = 1.0$ 、1.0、1.0（18 族内）也全不显著；游程检验问的是前后是否独立，

$n = 10^6$ 时 40/40，一击命中。同一个生成器，一种检验看不见，另一种看得清清楚楚。如果只做卡方，Short_LCG 会被误判为诚实——这就是测谎需要两道关的原因。

再看 Pi (H5)。它在三个 n 层的卡方拒绝率是 0.05、0.10、0.10，游程拒绝率是 0.10、0.05、0.125，全部贴着 α 波动；与 Py_random 的配对 McNemar 无一显著 ($p = 1.0$ 、0.375、0.375)。在本文的检测分辨率下， π 的数字与一把好骰子一样诚实。对高中生读者来说，这大概是全文最亲切的一格：圆周率数字通过了本文的两项检验。

再看诚实组的无罪判决 (H6)。没有被做手脚的这五家——C_rand、LCG_hand、Pi、Mersenne 与标杆 Py_random——拒绝率全部贴着 α 摆动。其中 C_rand、LCG_hand、Pi 与标杆 Py_random 的配对 McNemar 检验 (卡方口径)，在三个样本量下的 p 值分别是 0.625/1.0/0.375、1.0/1.0/1.0 与 1.0/0.375/0.375，最小的也有 0.375，离 $\alpha^* \approx 0.0028$ 的门槛远得很；Mersenne 与 Py_random 的配对不在预注册 18 族之内，作为族外描述性交叉验证， p 值为 0.625、0.625、0.375，同样不显著。诚实以”未被证伪”的形式成立。

取模演示 (H3) 的结果见表 3。C_rand 换成 $(x \bmod 6) + 1$ 之后，三个 n 层与统一映射的配对 McNemar 全部不显著 ($p = 1.0$ 、1.0、0.375)，预注册的零结果成立：RAND_MAX=32767 时取模偏差的非中心参数只有 $\lambda \approx 0.0075$ ， $n \leq 10^6$ 的样本量根本看不见。对照组 Low_LCG 的模数只有 32，分辨率太低，无论统一映射还是取模映射都 40/40 拒绝，统一映射下的对照基线 (H3a) 由此成立 (表 3)。分辨率决定可检测性，这正是第 3 节强调两种映射区别的原因。

表 3 取模演示：C_rand 与 Low_LCG 在取模映射 (H3b) 与统一映射 (H3a) 下的拒绝率 (C_rand 的统一映射数据见表 2)

生成器	映射	n	卡方 ρ	游程 ρ	与统一映射的 McNemar p
C_rand	取模映射	10^4	0.050	0.050	1.0
C_rand	取模映射	10^5	0.050	0.075	1.0
C_rand	取模映射	10^6	0.025	0.000	0.375
Low_LCG	取模映射	10^4	1.000	1.000	—
Low_LCG	取模映射	10^5	1.000	1.000	—
Low_LCG	取模映射	10^6	1.000	1.000	—
Low_LCG	统一映射	10^4	1.000	1.000	—
Low_LCG	统一映射	10^5	1.000	1.000	—
Low_LCG	统一映射	10^6	1.000	1.000	—

注：McNemar 列为 C_rand 在取模映射与统一映射同一种子上的配对比较；Low_LCG 的 McNemar 未纳入主比较族。C_rand 的零结果来自 RAND_MAX=32767 时 $\lambda \approx 0.0075$ ，见第 6 节。

图 2 把每面频数与 $n/6$ 之差的累积画成锯齿线。诚实的生成器围绕零来回摆动，振幅大致被随机游走的尺度框住；Biased_Low 的锯齿明显不对称，Low_LCG 干脆一路偏向一边。视觉上的歪，与统计上的拒绝互相印证。

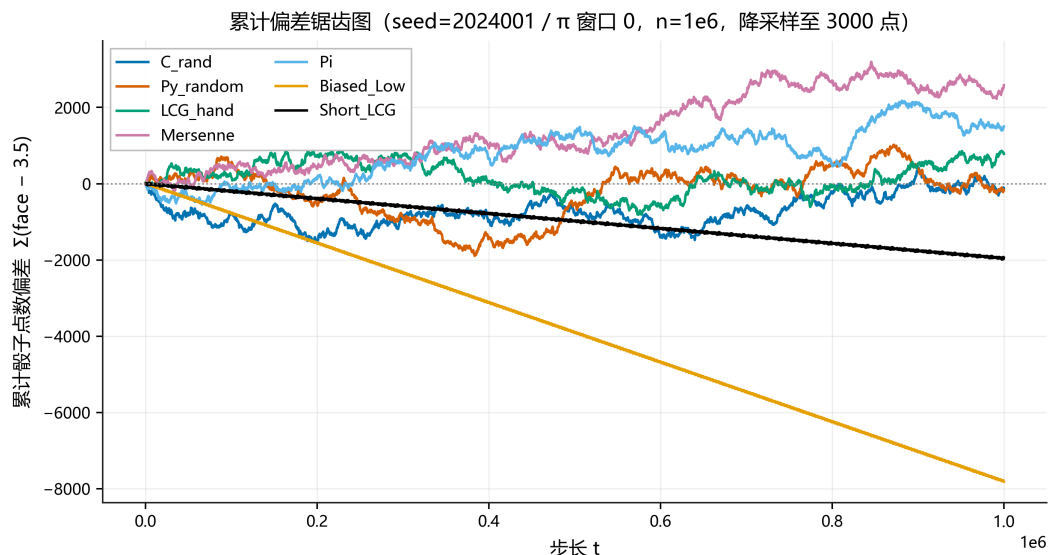


图 2 累计偏差锯齿图：各生成器逐面频数与期望频数之差的累积走势（诚实者绕零摆动，偏置者持续偏离）

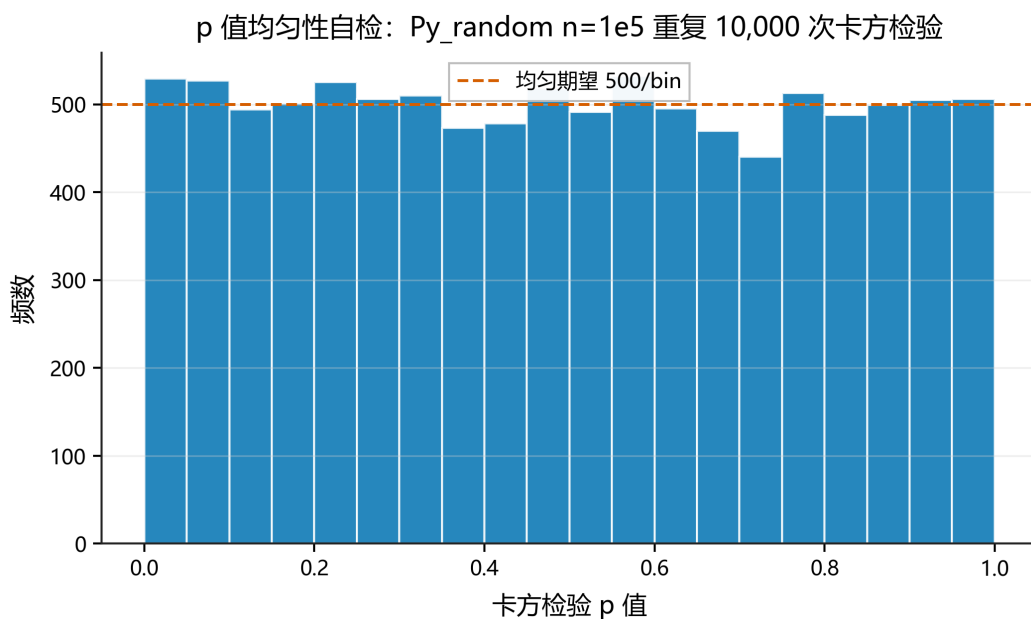


图 3 标杆 Py_random 的 10000 次卡方检验 p 值直方图：在 (0,1) 上近似平坦，证明检验实现无系统性偏差（前置闸门 G2）

图 3 是 p 值直方图。如果检验代码有 bug，p 值会偏向某一边，直方图就不再平坦。这里 10000 个 p 值近似均匀，配合前面 $\chi^2 = 19.820$ 、 $p = 0.4055$ 的拟合检验，代码的可信度有了双重证据。

复现本实验是容易的。实验脚本都放在 03_experiments/scripts/ 目录：runner.py 按配置网格逐条运行（840 次主矩阵加 360 次取模演示、40 个种子），aggregate.py 把逐次结果聚合为汇总表，make_plots.py 生成三张图，sanity_check.py 执行前置闸门，pvalue_uniformity.py 做 10000 次 p 值均匀性自检。 π 数字取自同一目录下的 pi_240m.txt。依序执行 python runner.py、python aggregate.py、python make_plots.py 即可重现本节全部结果；10000 次自检的命令是 python pvalue_uniformity.py --n 100000 --mc 10000。整个流程在本机约 10 秒内完成。

6 讨论与局限

为什么需要两种检验？因为卡方只看边缘分布，游程只看相邻关系，而作弊可以只动其中一种。`Short_LCG` 均匀但关联，所以卡方看不见、游程看得见；如果某个生成器只破坏均匀性而保持独立，情况就会反过来。两种检验像两道不同的安检门，缺一就会放行一种骗术。这正是大型套件 [14, 15] 存在的理由，也解释了为什么单项检验容易被突破 [18, 19]。

测谎是有分辨率的。任何检验都有看不见的下限。`C_rand` 取模偏差的非中心参数只有 $\lambda \approx 0.0075$ ，且随样本量线性增长 ($\lambda \approx 7.5 \times 10^{-9} n$)，要到 $n = 10^9$ 量级才升到 $\lambda \approx 7.5$ 、具备检测力；本文最大样本量是 10^6 ，所以它被判无罪。这不是放它过关，而是提醒读者，测谎结论必须和样本量一起读：检验的分辨率决定能识破什么级别的伪装。

结论的适用范围要如实说明。`C_rand` 的判定针对本机的 Windows MSVC 实现，`RAND_MAX=32767`，换一套运行时结果可能不同。`Mersenne` 在 $n = 10^5$ 、 $n = 10^6$ 时卡方拒绝率分别是 0.075 (3/40) 与 0.100 (4/40)，与 α 相差无几，配对比较与趋势检验都不显著，属于随机波动，不宜单格过度解读。此外，本文只检验了骰子面这一种粒度，更细的字节级、串行相关与谱检验 [8, 12] 都不在范围内；游程检验只盯一阶相邻关系，抓不住隔三差五的长程关联；40 个种子让拒绝率的颗粒是 0.025，想分辨 $\rho = 0.05$ 与 $\rho = 0.10$ 这类细差需要更多种子；Pi 的三个 n 层使用嵌套窗口，层与层并不完全独立。

未来可以做几件事。把种子加倍到 80 或 120，分辨率会从 0.025 降到 0.0125 或约 0.0083；把游程族正式纳入校正预算，让互补性结论有校正后的 p 值；引入串行相关与间隙检验，把测谎升级到第二档；再画一条检测力随样本量变化的曲线，把分辨率问题量化。对学习来说，把本实验的脚本跑一遍，改一改模数或映射，就是现成的探究项目。

7 结论

回到标题的问题：计算机真的能掷骰子吗？答案是，看情况。基于 1200 次运行的证据，本文给出三条具体判断。

第一，测谎仪是有用的。面对故意作弊的 `Biased_Low`，拒绝率从 $n = 10^4$ 的 0 跳到 $n \geq 10^5$ 的 1.00 (40/40)，配对 McNemar $p = 1.17 \times 10^{-9}$ ，Cochran–Armitage $z = 9.49$ ， $p = 2.4 \times 10^{-21}$ 。检验确实会响。

第二，测谎仪是有分工的。`Short_LCG` 让卡方检验 0/40 全数漏检，游程检验却在 $n = 10^6$ 一网打尽 40/40。单项检验要么冤枉好人，要么放过坏人，把均匀性检验与独立性检验并用，是给伪随机数生成器做测谎的最低配置。

第三，诚实不能被证明、只能被检验（注记 4.1）。`C_rand`、`LCG_hand`、`Mersenne`、Pi 与标杆 `Py_random` 的拒绝率全部贴着 α 波动；其中 `C_rand`、`LCG_hand`、Pi 与 `Py_random` 的配对比较在预注册 18 族内无一显著，`Mersenne` 与 `Py_random` 的配对属族外描述性交叉验证，同样不显著。在本文的检测分辨率下、40 次重复与预注册校正之后，这四家未能被证伪，与标杆 `Py_random` 相比无显著差异。 π 的小数位也因此与一把好骰子一样清白。

一句话收尾：统计学不能保证任何一把骰子永不作弊，但它能让作弊者在足够多的样本面前现形。检验的分辨率决定了我们能识破什么级别的伪装，而把均匀与独立这两道关都过一遍，就是计算机掷骰子的最低底线。

参考文献

- [1] K. Pearson. *On the criterion that a given system of deviations from the probable in the case of a correlated system of variables is such that it can be reasonably supposed to have arisen from random sampling*. Philosophical Magazine, Series 5, 50(302), 157–175, 1900.
- [2] R. A. Fisher. *On the interpretation of χ^2 from contingency tables, and the calculation of P* . Journal of the Royal Statistical Society, 85(1), 87–94, 1922.
- [3] E. B. Wilson. *Probable inference, the law of succession, and statistical inference*. Journal of the American Statistical Association, 22(158), 209–212, 1927.
- [4] A. Wald, J. Wolfowitz. *On a test whether two samples are from the same population*. Annals of Mathematical Statistics, 11(2), 147–162, 1940.
- [5] Q. McNemar. *Note on the sampling error of the difference between correlated proportions or percentages*. Psychometrika, 12(2), 153–157, 1947.
- [6] D. H. Lehmer. *Mathematical methods in large-scale computing units*. In Proceedings of the 2nd Symposium on Large-Scale Digital Calculating Machinery, 141–146, 1951.
- [7] W. G. Cochran. *Some methods for strengthening the common χ^2 tests*. Biometrics, 10(4), 417–451, 1954.
- [8] G. Marsaglia. *Random numbers fall mainly in the planes*. Proceedings of the National Academy of Sciences, 61(1), 25–28, 1968.
- [9] S. K. Park, K. W. Miller. *Random number generators: good ones are hard to find*. Communications of the ACM, 31(10), 1192–1201, 1988.
- [10] G. Marsaglia. *The Marsaglia Random Number CDROM including the Diehard Battery of Tests of Randomness*. 1995.
- [11] D. Bailey, P. Borwein, S. Plouffe. *On the rapid computation of various polylogarithmic constants*. Mathematics of Computation, 66(218), 903–913, 1997.
- [12] D. E. Knuth. *The Art of Computer Programming, Volume 2: Seminumerical Algorithms, 3rd edition*. Addison-Wesley, 1998.
- [13] M. Matsumoto, T. Nishimura. *Mersenne twister: a 623-dimensionally equidistributed uniform pseudo-random number generator*. ACM Transactions on Modeling and Computer Simulation, 8(1), 3–30, 1998.
- [14] P. L’Ecuyer, R. Simard. *TestU01: a C library for empirical testing of random number generators*. ACM Transactions on Mathematical Software, 33(4), Article 22, 2007.
- [15] A. Rukhin, J. Soto, J. Nechvatal, M. Smid, E. Barker, et al. *A Statistical Test Suite for Random and Pseudorandom Number Generators for Cryptographic Applications*. NIST Special Publication 800-22 Rev. 1a, 2010.

- [16] K. Okada, K. Endo, K. Yasuoka, et al. *Learned pseudo-random number generator: WGAN-GP for generating statistically robust random numbers*. PLoS ONE, 18(6), e0287025, 2023. <https://doi.org/10.1371/journal.pone.0287025>.
- [17] C. Foreman, R. Yeung, F. J. Curchod. *Statistical testing of random number generators and their improvement using randomness extraction*. Entropy, 26(12), 1053, 2024. <https://doi.org/10.3390/e26121053>.
- [18] P. Dahiya, I. Shumailov, R. Anderson. *Machine learning needs better randomness standards: Randomised smoothing and PRNG-based attacks*. In Proceedings of the 33rd USENIX Security Symposium, 2024.
- [19] E. Riesinger, J. Fuß. *Testing the Tests — Opportunities for Corrections and Improvements in NIST SP 800-22r1a and its Reference Code*. IACR Cryptology ePrint Archive 2025/856, 2025.
- [20] H. Chai, Q. Pan, J. Wu. *A survey of random number generator: Approaches, tests, novel applications in block-chain and AI driven industrial networks*. Security and Safety, 4, 2025015, 2025. <https://doi.org/10.1051/sands/2025015>.